

Univerzita Jana Evangelisty Purkyně
v Ústí nad Labem
Přírodovědecká fakulta



GoodAccess CLI klient pro Linux

BAKALÁŘSKÁ PRÁCE

Vypracoval: Valdemar Pospíšil

Vedoucí práce: Ing. Jiří Hromádka

Studijní program: Aplikovaná informatika

Studijní obor:

ÚSTÍ NAD LABEM 2025

Namísto žlutých stránek vložte digitálně podepsané zadání kvalifikační práce poskytnuté vedoucím katedry.

Zadání musí zaujímat právě dvě strany.

Zadání je nutno vložit jako PDF pomocí některého nástroje, který umožňuje editaci dokumentů (se zachováním elektronického podpisu).

V Linuxe lze například použít příkaz `pdftk`.

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně a použil jen pramenů, které cituji a uvádím v přiloženém seznamu literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., ve znění zákona č. 81/2005 Sb., autorský zákon, zejména se skutečností, že Univerzita Jana Evangelisty Purkyně v Ústí nad Labem má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Jana Evangelisty Purkyně v Ústí nad Labem oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladu, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

V Ústí nad Labem dne 7. prosince 2025

Podpis:

Děkuji vedoucímu práce **Ing. Jiřímu Hromádkovi**
za neocenitelné rady a pomoc při tvorbě bakalářské práce.

Abstrakt:

V současné době nabízí GoodAccess VPN řešení především prostřednictvím grafického uživatelského rozhraní, což nevyhovuje všem uživatelským scénářům. Zejména v prostředí serverů, automatizovaných systémů a DevOps pracovních postupů preferují administrátoři a pokročilí uživatelé řešení příkazové řádky.

Hlavním cílem této práce je navrhnout a implementovat funkční prototyp CLI aplikace pro správu GoodAccess VPN připojení na Linux distribucích s využitím WireGuard protokolu. Práce se zabývá návrhem architektury, bezpečným ukládáním dat a implementací komunikace se systémovou službou pomocí IPC.

Klíčová slova: VPN, GoodAccess, CLI, Linux, Go, .NET, WireGuard, IPC, Clean Architecture

Abstract:

Currently, GoodAccess VPN offers solutions primarily through a graphical user interface, which does not suit all user scenarios. Especially in server environments, automated systems, and DevOps workflows, administrators and advanced users prefer command-line solutions.

The main goal of this work is to design and implement a functional prototype of a CLI application for managing GoodAccess VPN connections on Linux distributions using the WireGuard protocol. The work deals with the design of the architecture, secure data storage, and implementation of communication with the system service using IPC.

Keywords: VPN, GoodAccess, CLI, Linux, Go, .NET, WireGuard, IPC, Clean Architecture

Obsah

Úvod	13
1. Teoretická východiska	15
1.1. Architektura a správa OS Linux	15
1.2. VPN Protokoly	16
1.3. Meziprocesová komunikace (IPC)	16
1.4. Platforma .NET na Linuxu	17
1.5. Metodika vývoje	17
2. Analýza a specifikace požadavků	19
2.1. Identifikace zúčastněných stran	19
2.2. Funkční požadavky	19
2.3. Nefunkční požadavky	20
2.4. Analýza architektury	20
3. Návrh řešení	23
3.1. Celková architektura systému	23
3.2. Návrh komunikace (IPC)	23
3.3. Návrh uživatelského rozhraní (CLI)	23
3.4. Bezpečnostní model	23
3.5. Datový model a konfigurace	23
4. Implementace	25
4.1. Struktura projektu a nástroje	25
4.2. Implementace klienta (Frontend)	25
4.3. Úpravy backendové služby	25
4.4. Integrace WireGuardu	25
4.5. Distribuční balíčky a Systemd	25
5. Testování a validace	27
5.1. Metodika testování	27
5.2. Jednotkové testy (Unit Testing)	27
5.3. Integrovaní a systémové testy	27
5.4. Testování na různých distribucích	27

Závěr	29
A. Externí přílohy	31
B. Další přílohy	33

Úvod

V posledních letech došlo k zásadnímu posunu v organizaci práce a správě síťové infrastruktury. S masivním přechodem na práci na dálku a distribuované týmy se tradiční modely zabezpečení sítě ukazují jako nedostatečné. Organizace stále častěji adoptují architektury typu Zero Trust Network Access (ZTNA), kde je bezpečné VPN připojení klíčovým prvkem nejen pro koncové uživatele, ale i pro serverovou infrastrukturu.

Platforma GoodAccess v současné době nabízí řešení primárně prostřednictvím grafického uživatelského rozhraní (GUI). Toto řešení je vhodné pro běžné uživatele na osobních počítačích, avšak naráží na limity v prostředí serverů, automatizovaných systémů a DevOps pracovních postupů.

Motivace

V serverovém prostředí Linux, které je dominantní platformou pro cloudovou infrastrukturu, často není grafické rozhraní dostupné (tzv. headless servery). Administrátoři a DevOps inženýři vyžadují nástroje, které lze ovládat z příkazové řádky, snadno skriptovat a integrovat do CI/CD pipeline. Absence nativního CLI (Command Line Interface) klienta pro Linux představuje pro GoodAccess značné omezení v možnosti nasazení v těchto scénářích.

Cíle práce

Hlavním cílem této bakalářské práce je navrhnout a implementovat funkční prototyp CLI aplikace pro správu GoodAccess VPN připojení na Linux distribucích. Aplikace bude navržena s důrazem na bezpečnost, modularitu a soulad se stávající infrastrukturou GoodAccess.

Dílčí cíle práce zahrnují:

- Analýzu specifik operačního systému Linux s ohledem na správu síťových rozhraní a systémových služeb.
- Návrh architektury řešení, která odděluje privilegovanou logiku služby od uživatelského rozhraní.
- Implementaci komunikace mezi klientem (Go) a backendovou službou (.NET) pomocí IPC.
- Integraci podpory pro protokol WireGuard a zajištění zpětné kompatibility.

Struktura práce

Práce je rozdělena do několika logických celků. Kapitola 1 se věnuje teoretickým východiskům, popisuje architekturu Linuxu, použité VPN protokoly a metody meziprocesové komunikace. Kapitola 2 definuje požadavky na systém a analyzuje možné architektonické přístupy. Následující kapitoly se poté věnují samotnému návrhu, implementaci a testování výsledného řešení.

1. Teoretická východiska

Tato kapitola shrnuje klíčové technologie a koncepty nezbytné pro návrh a implementaci systémového CLI nástroje pro Linux. Zaměřuje se na architekturu operačního systému, síťové protokoly a principy vývoje softwaru.

1.1. Architektura a správa OS Linux

Pro vývoj robustního systémového nástroje je nezbytné pochopení interakce mezi uživatelským prostorem (User Space) a jádrem (Kernel). Jak uvádí Ward [**ward_linux**], jádro operačního systému spravuje hardwarové prostředky a poskytuje abstrakci pro aplikace běžící v uživatelském prostoru.

Jádro vs. Uživatelský prostor

Zatímco vyvíjený CLI klient bude běžet v uživatelském prostoru, samotná síťová rozhraní (Network Interfaces) jsou spravována jádrem. Komunikace mezi těmito vrstvami probíhá prostřednictvím systémových volání (syscalls). Pro manipulaci se síťovým zásobníkem (networking stack) Linux využívá rodinu protokolů Netlink, kterou využívají i standardní nástroje jako `ip`.

Virtuální souborové systémy

Linux zpřístupňuje stav jádra a hardwaru prostřednictvím virtuálních souborových systémů `/proc` a `/sys`. Tyto adresáře neobsahují data na disku, ale slouží jako rozhraní k datovým strukturám jádra v paměti. Pro CLI klienta jsou tato místa klíčová pro získávání statistik o síťovém provozu a stavu tunelů.

Model oprávnění a Systemd

Správa VPN připojení vyžaduje privilegované operace, jako je úprava routovacích tabulek nebo konfigurace DNS. Linuxový bezpečnostní model striktně odděluje běžné uživatele od uživatele `root`. Aplikace bude využívat principu nejnižších privilegií (Least Privilege), kdy samotný CLI klient běží pod běžným uživatelem a komunikuje s privilegovanou službou (daemonem).

Pro správu této služby na pozadí je v moderních distribucích (Debian, RHEL, Fedora) standardem init systém **systemd**. Ten umožňuje definovat závislosti (např. spuštění VPN až po

`network-online.target`) a zajišťuje automatický start služby po restartu systému pomocí konfiguračních souborů zvaných *Unit files*.

1.2. VPN Protokoly

GoodAccess podporuje více protokolů pro zajištění konektivity. Ačkoliv je moderním standardem WireGuard, je nutné brát v úvahu i existenci staršího protokolu OpenVPN.

WireGuard

WireGuard je moderní VPN protokol navržený s důrazem na jednoduchost, vysoký výkon a bezpečnost. Podle Donenfelda [[wireguard_paper](#)] je jeho klíčovou vlastností běh přímo v prostoru jádra (Kernel Space), což eliminuje nákladné přepínání kontextu (context switching) typické pro starší řešení.

Cryptokey Routing: Fundamentálním konceptem WireGuardu je směrování založené na kryptografických klíších. Každý peer má asociovaný veřejný klíč a povolenou IP adresu. Toto statické provázání zjednodušuje konfiguraci a zvyšuje bezpečnost.

Vlastnost ticha (Silence Property): WireGuard je navržen tak, aby neodpovídal na neautentizované pakety. Pokud klient nepošle platná data, server mlčí. To stěžuje detekci VPN endpointů pomocí skenování portů, ale zároveň klade nároky na implementaci klienta, který musí stav připojení odvozovat nepřímou (např. pomocí handshake timerů), nikoliv ze stavu TCP spojení.

OpenVPN

OpenVPN je zavedený, flexibilní protokol založený na SSL/TLS. Na rozdíl od WireGuardu běží tradičně v uživatelském prostoru (User Space) a využívá virtuální síťová rozhraní TUN/TAP. Ačkoliv je OpenVPN vysoce konfigurovatelný, jeho architektura přináší vyšší režii (overhead) způsobenou nutností kopírovat data mezi jádrem a uživatelským prostorem při každém paketu. V rámci této práce je OpenVPN zmíněn jako existující alternativa implementovaná v backendové službě, kterou musí CLI klient umět v případě potřeby aktivovat.

1.3. Meziprocesová komunikace (IPC)

Zvolená architektura odděleného klienta a služby vyžaduje efektivní komunikační kanál.

Unix Domain Sockets

Pro lokální komunikaci na Linuxu jsou standardem Unix Domain Sockets. Na rozdíl od TCP socketů nevyužívají síťový stack (loopback), ale souborový systém, což zvyšuje efektivitu přenosu. Klíčovou

výhodou je bezpečnost – přístup k socketu lze řídit standardními souborovými oprávněními (např. ‘chmod 660’), což zamezuje přístupu neoprávněným uživatelům na stejném stroji.

Serializace dat

Data přenášená přes IPC musí být serializována.

- **JSON:** Textový formát, snadno čitelný pro člověka, ideální pro ladění. Nevýhodou je nižší výkon a absence striktní typové kontroly.
- **gRPC / Protocol Buffers:** Binární formát vyvíjený Googlem. Poskytuje silnou typovou kontrolu, definici kontraktu (interface definition) a vysoký výkon. Vzhledem ke kombinaci jazyků Go a C# (.NET) je gRPC silným kandidátem pro zajištění typové bezpečnosti mezi procesy.

1.4. Platforma .NET na Linuxu

Backendová služba GoodAccess je postavena na platformě .NET 8. Jak uvádí Price [**price_csharp**], moderní .NET (od verze Core) je plně multiplatformní. Pro účely této práce je klíčová schopnost .NET aplikací běžet jako nativní linuxoví démoni (Systemd Service) a využívat mechanismy P/Invoke pro volání nativních knihoven systému, což je nezbytné pro konfiguraci síťových rozhraní.

1.5. Metodika vývoje

Při vývoji jsou uplatňovány principy Extrémního programování (XP) podle Becka [**beck_xp**]. Důraz je kladen na **jednoduchost** (návrh nejjednoduššího možného řešení, které funguje) a **zpětnou vazbu** (krátké iterace, okamžitá odezva CLI aplikace).

2. Analýza a specifikace požadavků

Cílem této kapitoly je analyzovat potřeby cílové skupiny uživatelů a na jejich základě definovat funkční i nefunkční požadavky na výslednou aplikaci. Součástí je také analýza možných architektonických přístupů.

2.1. Identifikace zúčastněných stran

Primárními uživateli GoodAccess CLI pro Linux jsou technicky zdatní profesionálové:

- **System Administrators:** Spravují servery a vyžadují stabilní nástroj pro vzdálenou správu přes SSH bez nutnosti grafického rozhraní.
- **DevOps Inženýři:** Potřebují integrovat VPN připojení do automatizovaných skriptů a CI/CD pipeline. Vyžadují predikovatelné chování a strojově čitelný výstup.

2.2. Funkční požadavky

Na základě analýzy potřeb byly stanoveny následující funkční požadavky:

Autentizace a správa relace

Aplikace musí umožnit přihlášení uživatele v prostředí bez webového prohlížeče.

- **F1 – Login:** Podpora pro autentizační toky vhodné pro terminál (např. Device Code Flow), kdy uživatel potvrdí přihlášení na jiném zařízení.
- **F2 – Logout:** Bezpečné ukončení relace a smazání lokálně uložených tokenů.

Správa připojení

Jádrum aplikace je ovládání VPN tunelu.

- **F3 – Connect:** Navázání připojení k vybrané bráně. Aplikace musí podporovat výběr protokolu (WireGuard jako výchozí, OpenVPN jako záložní).
- **F4 – Disconnect:** Korektní ukončení VPN připojení a úklid routovacích pravidel.

- **F5 – Gateway Selection:** Uživatel musí mít možnost vybrat si, ke které VPN bráně (Gateway) se chce připojit.

Monitoring a stav

- **F6 – Status:** Zobrazení aktuálního stavu (Připojeno/Odpojeno, IP adresa, přenesená data).
- **F7 – JSON Output:** Možnost vyžádat si výstup příkazů (zejména status) ve formátu JSON pro snadné parsování v externích skriptech.

2.3. Nefunkční požadavky

- **N1 – Výkon:** Rychlý start aplikace (CLI). Jazyk Go byl zvolen právě pro svou rychlost spouštění a malou paměťovou náročnost.
- **N2 – Kompatibilita:** Aplikace musí být funkční na hlavních serverových distribucích Linuxu (Debian/Ubuntu, Fedora/RHEL, Arch Linux).
- **N3 – Bezpečnost:** Citlivá data (přístupové tokeny, privátní klíče) musí být uložena bezpečně a přístupná pouze oprávněné službě.

2.4. Analýza architektury

Při návrhu řešení byly zvažovány dva přístupy k interakci s existující logikou GoodAccess:

Návrh A: Tenký klient (Wrapper)

V tomto modelu běží v systému jedna privilegovaná služba (.NET), která drží stav a vykonává operace. CLI aplikace (Go) slouží pouze jako dálkové ovládání, které posílá příkazy službě přes IPC.

Návrh B: Samostatná logika

CLI aplikace by obsahovala veškerou logiku pro připojení a správu WireGuardu a běžela by přímo pod uživatelem root, nezávisle na existující službě.

Zvolené řešení

Byl zvolen **Návrh A (Wrapper)** z následujících důvodů:

1. **Single Source of Truth:** Služba drží jednotný stav aplikace. Nedochází k desynchronizaci mezi tím, co si myslí CLI, a tím, co je reálně nastaveno v síti.
2. **Bezpečnost:** Uživatel nemusí spouštět CLI pod rootem. Privilegované operace provádí pouze izolovaná služba na pozadí.
3. **Znovupoužitelnost:** Využívá se již otestovaná a existující logika v .NET backendu, což je v souladu s principy XP (Simplicity).

3. Návrh řešení

V této kapitole je představen detailní technický návrh aplikace. Vychází z analytických požadavků a zvolené architektury tenkého klienta.

3.1. Celková architektura systému

3.2. Návrh komunikace (IPC)

3.3. Návrh uživatelského rozhraní (CLI)

3.4. Bezpečnostní model

3.5. Datový model a konfigurace

4. Implementace

Kapitola popisuje proces vývoje, použitou strukturu projektu a klíčové implementační detaily jednotlivých komponent.

4.1. Struktura projektu a nástroje

4.2. Implementace klienta (Frontend)

4.3. Úpravy backendové služby

4.4. Integrace WireGuardu

4.5. Distribuční balíčky a Systemd

5. Testování a validace

Ověření funkčnosti a stability aplikace probíhalo v souladu s metodikou Extrémního programování průběžně během vývoje.

5.1. Metodika testování

5.2. Jednotkové testy (Unit Testing)

5.3. Integrační a systémové testy

5.4. Testování na různých distribucích

Závěr

Cílem této bakalářské práce byl návrh a implementace nativního CLI klienta GoodAccess pro OS Linux.

Shrnutí výsledků

Diskuse a přínos

Možnosti budoucího rozvoje

A. Externí přílohy

Externí přílohy této bakalářské práce jsou umístěny na adrese:

https://github.com/Jiri-Fiser/thesis_ki_ujep.

Na úložišti GitHub mohou být uloženy tyto externí přílohy:

- **zdrojové kódy**
- **doplňkové texty** (například jak instalovat aplikaci, manuály aplikace)
- **schémata** (především, pokud se nevejdou na stranu A4 a jejich vtištění je tak problematické)
- **screenshoty** (v textu práce lze použít jen omezený počet snímků obrazovky, které navíc nemusí být při černobílém tisku příliš přehledné)
- **videa** (například ovládání aplikace)

V každém případě by to však měli být pouze materiály, které jste vytvořili sami. Materiály jiných autorů uvádějte v seznamu použité literatury (včetně případných odkazů na jejich originální umístění).

V této kapitole stačí uvést pouze základní strukturu úložiště (co se kde nalézá a jakou má funkci) například v podobě tabulky.

ki-thesis.pdf	text práce v PDF
ki-thesis.tex	zdrojový kód práce v $\text{\LaTeX}$
kitheses.cls	definice třídy dokumentů (rozšířená třída scrbook)
thesis.bib	bibliografická databáze (exportována z citace.com)
LOGO_PRF_CZ_RGB_standard.jpg	logo fakulty s českým textem
LOGO_PRF_EM_RGB_standard.jpg	logo fakulty s anglickým textem

Všechny tyto soubory jsou potřeba pro překlad dokumentu (logo stačí jedno v příslušné jazykové verzi).

B. Další přílohy

Výjimečně může práce obsahovat i další tištěné přílohy. Obecně však dávejte přednost elektronickým přílohám umístěným na GitHubu (tato kapitola tak bude úplně chybět).